

CODE SPORT



In this month's column, we discuss a few computer science interview questions.

1. Containers are becoming the *de facto* method for application deployment, moving away from virtual machines. Docker is a popular container application which most of you would be familiar with (<https://www.docker.com/>). Docker is an open source project built on top of Linux containers, which allows applications to be isolated from each other. Can you explain how Docker differs in providing isolation to applications as compared to virtual machines? Can you compare the two methods of application deployment, namely, hosting an application on a VM vs hosting it on a container, with regard to the following criteria: (a) security, (b) isolation, (c) cost, and (d) fine grained resource control and sharing.
2. When we input a query into a search engine such as Google, we get an 'auto complete' suggestion for that query, based on past queries that have been tried on the search engine. Can you explain how you would design an auto-complete mechanism that would scale well across the millions of queries that can happen concurrently at a time? What is the data structure that you would use to store past queries and automatically provide suggestions? Would you store the complete query as is, or would you break it into its constituent words? Given that scalability and correctness is the key to this problem, can you explain how the choice of your data structure will address both?
3. In information retrieval, you are given a set of documents and a query. You are asked to retrieve the set of documents that are most relevant to the query. Many of the common information retrieval techniques are built around the measure of TF-IDF, where TF stands for the term frequency, i.e., the number of times a query keyword appears in a document. For example, given the query 'Mahendra Singh Dhoni's test match appearances', I am asked to find all the documents relevant to it. It is obvious that one can compute the Term Frequency for the query terms, and return a ranked list of documents in which the query terms appear most frequently. If this suffices to return the most relevant documents, why do we need the measure of Inverse Document Frequency (IDF)? Can you illustrate, with an example, why IDF is needed in information retrieval? If you ignore IDF for relevant document retrieval, would it impact the precision or recall, or both?
4. With elections coming around in five Indian states, poll predictions have become quite a hot topic. However, there have been mixed results for poll predictions. For instance, the past year has been quite unsuccessful for many of the data scientists who are into poll prediction. Neither Brexit nor Trump's victory were anticipated by most pollsters. Given this context, you are asked to predict the party that will win the majority in each of the five states going to the polls in the coming months. For any prediction, you need to build models which can analyse the data and make the prediction. What are the various data inputs that you would be interested in getting? Or, in simplistic terms, what are some of the obvious features that you would like to extract from the data for winning party predictions in a state election? Is this a classification problem or a clustering problem? Why do you think poll prediction is a hard problem?
5. You are given a set of documents and asked to perform clustering of these documents based on document level similarity. You are familiar with different clustering algorithms such as 'kmeans', so you are confident that you can do the task. However, the interviewer now adds a second part to the problem, wanting you to label the clusters. How would you generate labelled clusters? What are the means of generating meaningful labels for the clusters? Can the clusters be labelled after, prior to, or during the clustering process itself? Describe your choice of solution and why you made it.